

AssetFlow — Backend & API PRD

Scope: Backend Engineer's Track (Node.js + Express + TypeScript + Prisma)

Context: This is a team hackathon build. Database schema is owned by the DB teammate (Prisma schema already exists / is being finalized in parallel). Frontend is owned by the frontend teammate. **This document scopes ONLY the backend service: APIs, business logic, auth, RBAC, workflows, validation, security, real-time events, and testing.** Written so Claude Code can implement directly with minimal ambiguity.

Assumption: The Prisma schema described in §1 (Data Contract) is the agreed contract with the DB teammate. If the actual schema differs, treat §1 as the source of truth to build against and flag drift — do not silently redesign it.

1. Data Contract (consumed, not owned)

The backend assumes these Prisma models exist (own copy for reference/type-safety; do not edit schema.prisma unless DB teammate is unavailable):

- `User` (id, name, email, passwordHash, role, departmentId, isActive)
- `Department` (id, name, parentId, isActive)
- `AssetCategory` (id, name, customFields Json)
- `Asset` (id, assetTag, name, categoryId, serialNumber, purchaseDate, purchaseCost, condition, status, currentHolderId, customFields Json, isDeleted)
- `AssetHistory` (id, assetId, action, fromUserId, toUserId, notes, createdAt) — append-only
- `AllocationRequest` (id, assetId, type, requesterId, currentHolderId, approverId, status, reason, resolvedAt)

- `MaintenanceTicket` (id, assetId, raisedById, issueDesc, status, approverNotes, resolvedAt)
- `MeetingRoom` (id, name, capacity, location, isActive)
- `ResourceBooking` (id, roomId, bookedById, startTime, endTime, status, purpose)
- `AuditRecord` (id, performedById, startedAt, completedAt)
- `AuditItem` (id, auditId, assetId, result, notes)
- `Notification` (id, userId, type, message, isRead)
- `ActivityLog` (id, userId, action, entityType, entityId, metadata Json, createdAt) — append-only

Enums: `Role`, `AssetStatus`, `AssetCondition`, `RequestStatus`, `RequestType`, `MaintenanceStatus`, `BookingStatus`, `AuditItemStatus`, `NotificationType` (values as defined in the master PRD — pull exact enum values from the live schema.prisma before coding, don't hardcode from memory).

Backend responsibility boundary: you write `prisma/schema.prisma` migrations only if blocking; otherwise `npx prisma generate` against the DB teammate's schema and build entirely against the generated client + types.

2. Backend Deliverable Definition

A standalone Express + TypeScript API service that:

1. Exposes a fully working REST API per §6, matching the response envelope in §5.
2. Owns all business rules, state machines, and validation (frontend must not be trusted for any of this).
3. Emits Socket.IO events for real-time consumers (dashboard, notifications).
4. Is independently testable via a Postman collection, without requiring the frontend to exist.

5. Runs standalone via `npm run dev` against a `DATABASE_URL` env var, and deploys to Render via `npm run build && npm start`.

3. Tech Stack (backend slice)

- Node.js + Express.js + TypeScript
 - Prisma ORM (client only – schema owned by DB teammate)
 - JWT (`jsonwebtoken`) + `bcrypt` for auth
 - Zod for request validation
 - Helmet, express-rate-limit, CORS
 - Socket.IO server
 - Postman for endpoint testing (export collection to `/postman`)
 - dotenv for config; no secrets hardcoded
-

4. Architecture & Folder Structure

Modular, layered architecture – **controller** → **service** → **prisma**, never let controllers touch Prisma directly, never let services touch `req / res`.

```
backend/
├── src/
│   ├── config/
│   │   ├── env.ts           (zod-validated env
│   │   │                   schema, fail fast on missing vars)
│   │   ├── prisma.ts       (singleton PrismaClient)
│   │   └── socket.ts        (Socket.IO server init +
│   │                       auth handshake)
│   ├── middleware/
│   │   ├── auth.middleware.ts (verify JWT, attach
│   │   │                   req.user)
│   │   ├── rbac.middleware.ts
│   │   │   (requireRole(...roles))
│   │   ├── validate.middleware.ts (zod schema → 400
│   │   │                   on failure)
│   │   └── rateLimit.middleware.ts
```

```

| | | └─ error.middleware.ts      (centralized, last
in chain)
| | └─ modules/
| |   └─ auth/                    (signup, login, refresh,
logout)
| |   └─ departments/
| |   └─ categories/
| |   └─ employees/              (directory + role promotion)
| |   └─ assets/                (registration, tag
generation, history)
| |   └─ allocation/            (allocate, transfer, return
workflows)
| |   └─ bookings/              (room booking + conflict
detection)
| |   └─ maintenance/
| |   └─ audits/
| |   └─ reports/
| |   └─ notifications/
| |   └─ activityLogs/
| |       # each module contains:
| |       # *.routes.ts         - route wiring only
| |       # *.controller.ts     - req/res, calls service,
no business logic
| |       # *.service.ts        - business logic,
transactions, state machines
| |       # *.schema.ts         - Zod input/output
schemas
| |       # *.types.ts          - module-local types
| |   └─ utils/
| |   └─ apiResponse.ts         (success/error envelope
helpers)
| |   └─ asyncHandler.ts        (wraps async controllers,
forwards errors)
| |   └─ assetTag.ts            (sequential tag generator)
| |   └─ logger.ts
| |   └─ activityLogger.ts      (helper: write ActivityLog
from any service)
| |   └─ sockets/
| |   └─ events.ts              (typed event names + emit
helpers)
| |   └─ app.ts                 (express app, middleware
wiring, route mounting)
| |   └─ server.ts              (http server + socket.io

```

```
bootstrap, listen)
├─ prisma/ (reference copy / generated client – owned upstream)
├─ postman/AssetFlow-Backend.postman_collection.json
├─ .env.example
└─ package.json
```

Rule: no module's service function may be longer than it needs to be to express one business operation — decompose into private helpers within the service file rather than bloating controllers.

5. API Response Contract (must be identical across all endpoints)

```
// success
{ success: true, data: T, message?: string }

// failure
{ success: false, error: { code: string, message: string, details?: unknown } }
```

- HTTP status codes must match semantics: 200 OK, 201 Created, 400 validation, 401 unauthenticated, 403 forbidden (role/ownership), 404 not found, 409 conflict (double allocation, booking overlap, duplicate email), 500 only for truly unexpected errors (never leak stack traces — log server-side via `logger.ts`, return a generic message to the client).
 - Pagination convention for list endpoints: `?page=1&limit=20`, response `data: { items: T[], total, page, limit }.`
-

6. Endpoint Specification

6.1 Auth (`/api/auth`)

Method	Route	Body	Notes
POST	/signup	name, email, password, departmentId?	Force <code>role = EMPLOYEE</code> server-side regardless of any <code>role</code> field sent — strip it before validation even reaches the DB layer
POST	/login	email, password	Returns <code>{ accessToken, user }</code> , sets <code>httpOnly</code> refresh cookie
POST	/refresh	(cookie)	Rotates access token, validates refresh token signature + optional DB session check
POST	/logout	(cookie)	Clears refresh cookie
GET	/me	(auth)	Returns current user profile (safe fields only)

Business rules:

- Reject signup if email already exists → `409`.
- `bcrypt` cost factor ≥ 10 .
- Access token TTL ~15 min, refresh TTL ~7 days, both signed with separate secrets from env.
- Never return `passwordHash` in any response — use a `toSafeUser()` mapper everywhere a `User` is serialized.

6.2 Departments (`/api/departments`) — Admin only for writes

- `GET /` — list with hierarchy (nested or flat + `parentId`, frontend decides rendering).
- `POST /` — create (name, `parentId?`).
- `PATCH /:id` — edit / toggle `isActive`.

- Validation: prevent circular parent references (a department cannot be its own ancestor) — this is backend's job to catch.

6.3 Categories (/api/categories) — Admin only for writes

- CRUD + `customFields` as a schema definition (array of `{ key, label, type }`) stored in `Json` .
- On asset creation, validate submitted `customFields` values against the category's field definitions (type-check at minimum).

6.4 Employees (/api/employees) — Admin only

- `GET /` — list/search/filter by department, role, active status, paginated.
- `PATCH /:id/role` — the **only** endpoint in the entire system that can change `role` . Must:
 - Reject if actor is not `ADMIN` .
 - Reject self-demotion of the last remaining Admin (prevent logout).
 - Write an `ActivityLog` entry with old role → new role.
 - Emit a `Notification` to the affected user.

6.5 Assets (/api/assets)

- `POST /` (Asset Manager/Admin) — register asset:
 - Generate `assetTag` sequentially and atomically (use a DB transaction with `SELECT ... FOR UPDATE` on a counter row, or a Postgres sequence — **never** `COUNT(*) + 1` , it races under concurrent writes).
 - Write `AssetHistory` entry `"REGISTERED"` .
- `GET /` — list with filters (status, category, department of current holder, search by tag/name/serial), paginated.
- `GET /:id` — full detail.
- `GET /:id/history` — timeline of `AssetHistory` , newest first.
- `PATCH /:id` (Asset Manager/Admin) — edit metadata (not `status` — status only changes via workflow endpoints below).

- `DELETE /:id` (Admin) — soft delete (`isDeleted = true`), only if not currently `ALLOCATED`.

6.6 Allocation Workflow (`/api/allocation-requests`)

Core differentiator module — build carefully.

- `POST /` — body: `{ assetId, type: ALLOCATION|TRANSFER|RETURN, targetUserId?, reason? }`
 - **Allocation:** inside a DB transaction, re-check `Asset.status === AVAILABLE`. If not, return `409` with message `"Asset is already allocated to {holderName}"`. If available, either auto-approve (if requester is Manager/Admin) or create `PENDING` request for manager approval — confirm which flow with team, default to **requires manager approval** for MVP consistency.
 - **Transfer:** requires an existing holder; creates `PENDING` request; only `ASSET_MANAGER / ADMIN` can approve.
 - **Return:** holder or manager can initiate; on approval, asset condition is recorded, `status` → `AVAILABLE` (or `UNDER_MAINTENANCE` if condition marked `DAMAGED`).
- `PATCH /:id/approve` (Manager/Admin) — transactionally: update `Asset.status + currentHolderId`, write `AssetHistory`, mark request `APPROVED`, emit `Notification` to requester + previous holder (if transfer), emit socket event `asset:updated`.
- `PATCH /:id/reject` (Manager/Admin) — mark `REJECTED` with reason, notify requester.
- `GET /` — list, filterable by status/type/asset/user, paginated.

Concurrency requirement: the availability check + status update must happen inside a single Prisma `$transaction` to prevent race conditions from two simultaneous approval calls.

6.7 Bookings (`/api/bookings`)

- `GET /rooms` — list active meeting rooms.

- `POST /bookings` — body: { roomId, startTime, endTime, purpose? }
 - Overlap check (in transaction): reject if any existing `BOOKED` booking for the same room satisfies `existing.startTime < newEnd AND existing.endTime > newStart`. Return `409` with the conflicting slot info.
 - On success, emit socket event `booking:created` (so other clients viewing that room's calendar update live).
- `DELETE /bookings/:id` — owner or Admin only; sets `status = CANCELLED` (don't hard-delete, preserves history).
- `GET /bookings?roomId=&date=` — for calendar view.

6.8 Maintenance (/api/maintenance)

- `POST /` — any authenticated user raises a ticket on an asset they currently hold (or Manager on any asset). Status starts `REQUESTED`.
- `PATCH /:id/approve` (Manager/Admin) — `status → APPROVED / IN_PROGRESS`, `Asset.status → UNDER_MAINTENANCE`, write `AssetHistory`.
- `PATCH /:id/reject` (Manager/Admin) — `status → REJECTED`, notify raiser with reason.
- `PATCH /:id/complete` (Manager/Admin) — `status → COMPLETED`, `Asset.status → AVAILABLE`, write `AssetHistory`.
- `GET /` — list/filter by status/asset, paginated.

6.9 Audits (/api/audits)

- `POST /` (Manager/Admin) — start an `AuditRecord`, auto-populate `AuditItem` rows for every non-deleted asset in scope (all, or filtered by category/department if a `scope` param is passed).
- `PATCH /:auditId/items/:itemId` — record `FOUND | MISSING | DAMAGED` + notes.
- `POST /:auditId/complete` — set `completedAt`; for every item marked `MISSING`, transactionally update the linked `Asset.status → LOST` and write `AssetHistory`; generate a discrepancy

summary in the response ({ found, missing, damaged, total }).

- GET /:auditId — full report.

6.10 Reports (/api/reports) — Manager/Admin

- GET /most-used — assets ranked by allocation count/duration (aggregate query over AssetHistory / AllocationRequest).
- GET /least-used — inverse.
- GET /department-usage — asset counts grouped by current holder's department.
- GET /maintenance-forecast — assets with upcoming/recurring maintenance patterns (simple heuristic acceptable for MVP: assets with 2+ maintenance tickets in last N days).
- All report endpoints should be pure read/aggregate — no side effects, cache-friendly.

6.11 Notifications (/api/notifications)

- GET / — current user's notifications, paginated, ? unreadOnly=true filter.
- PATCH /:id/read — mark single read.
- PATCH /read-all — mark all read.
- Notifications are created server-side only, as a side effect of other module actions (never a public "create notification" endpoint) — implement via a shared notificationService.notify(userId, type, message) helper called from other services.

6.12 Activity Logs (/api/activity-logs) — Admin only

- GET / — filterable by entityType, entityId, userId, date range, paginated.
- Writes happen via a shared activityLogger.log(...) helper invoked from every state-changing service — do not rely on a global middleware to infer this generically; be explicit per action so action / entityType values stay meaningful (e.g. "ASSET_ALLOCATED", "ROLE_CHANGED", "BOOKING_CANCELLED").

6.13 Dashboard (/api/dashboard/summary)

- Role-aware aggregate: Employee gets counts scoped to self; Manager/Admin gets org-wide.
 - Returns: `availableAssets`, `allocatedAssets`, `maintenanceToday`, `roomsBookedToday`, `overdueReturns`.
 - Should be a single efficient query set (use `Promise.all` of count queries, not N+1).
-

7. Business Rules & State Machines (backend-enforced, non-negotiable)

Role elevation: only via `PATCH /employees/:id/role`, only by `ADMIN`. Every other endpoint must strip/ignore any `role` field in the request body.

Asset status transitions — implement as a single guarded function, e.g. `assetService.transitionStatus(assetId, newStatus, context)`, called from every module that changes asset state. Reject invalid transitions with `400`:

```
AVAILABLE → ALLOCATED | UNDER_MAINTENANCE | LOST |
RETIRED
ALLOCATED → AVAILABLE (via return/transfer completion)
UNDER_MAINTENANCE → AVAILABLE
RETIRED → DISPOSED
LOST → (terminal, requires manual Admin override if ever
recovered – separate explicit endpoint, not part of MVP
unless time allows)
```

No double allocation: enforced via transaction + status check, not via unique constraints alone (a `currentHolderId` unique-per-asset is implicit in the model, but the *check-then-act* must be atomic).

No booking overlap: enforced via transaction + overlap query, per room.

Approval workflows always follow: `requested` → (`approved` | `rejected`), and any status-changing approval must be wrapped in a Prisma `$transaction` covering: the request status update + the entity status update + the `AssetHistory` /audit trail write + notification creation. All-or-nothing.

8. Real-Time (Socket.IO) Contract

Namespace: default. Auth: client sends JWT on connection handshake (`socket.handshake.auth.token`), server verifies before allowing join.

Rooms: join a room per `userId` (for personal notifications) and optionally a global `dashboard` room for admins/managers.

Event	Emitted when	Payload
<code>notification:new</code>	Any <code>notificationService.notify()</code> call	<code>{ id, type, message, createdAt }</code>
<code>asset:updated</code>	Status change (allocation, return, maintenance, audit)	<code>{ assetId, status }</code>
<code>booking:created</code> / <code>booking:cancelled</code>	Room booking mutation	<code>{ roomId, booking }</code>
<code>dashboard:refresh</code>	Any mutation affecting dashboard counters	<code>{}</code> (client refetches summary)

Keep the emit calls inside services (not controllers) right after the DB transaction commits successfully — never emit before the write is confirmed.

9. Validation Strategy (Zod)

- One schema file per module (`*.schema.ts`), exporting `createXSchema` , `updateXSchema` , `queryXSchema` .
 - `validate.middleware.ts` takes a schema + target (`body` / `query` / `params`) and rejects with `400` + field-level errors on failure, before the controller runs.
 - Reject unknown/extra fields (`.strict()` on Zod objects) — this is what prevents a client from sneaking a `role` field into a signup payload.
-

10. Security Checklist (backend-owned)

- [] Helmet applied globally in `app.ts` .
 - [] CORS locked to the frontend's deployed origin + localhost dev origin via env var.
 - [] `express-rate-limit` : strict on `/api/auth/*` (e.g. 10 req/min), moderate globally (e.g. 100 req/min).
 - [] JWT secrets, DB URL, and all config pulled from `.env` , validated at boot via `config/env.ts` (fail fast, don't start server with missing/invalid env).
 - [] `toSafeUser()` (or Prisma `select`) used everywhere a `User` is returned — never leak `passwordHash` .
 - [] Centralized `error.middleware.ts` — catches all thrown errors, logs full detail server-side, returns sanitized message to client, never a raw stack trace.
 - [] RBAC middleware applied per-route, not assumed from frontend hiding UI elements.
 - [] All list endpoints paginated (no unbounded `findMany`).
 - [] Prisma used exclusively — no raw SQL string interpolation.
-

11. Testing Deliverable

- Postman collection (`/postman/AssetFlow-Backend.postman_collection.json`) covering every endpoint in §6, including at least one **failure-path** test per module (e.g. double allocation → 409 , booking overlap → 409 , non-admin hitting `/employees/:id/role` → 403 , invalid signup payload → 400).
 - Include a Postman environment file with `{{baseUrl}}` , `{{accessToken}}` variables and a pre-request script or documented manual step to log in and set the token.
 - A `seed.ts` script (coordinate with DB teammate) that creates: 1 Admin, 2 departments, 3 categories, 5 sample assets, 2 employees – needed for both Postman testing and frontend integration testing.
-

12. Interfaces Owed to Other Tracks

To Frontend teammate:

- Finalized API contract (this document, §5–§6) – treat as the integration spec; flag any endpoint changes immediately.
- Socket.IO event contract (§8).
- A running local instance + Postman collection they can cross-check against.

To DB teammate:

- Confirmation of exact enum values and any additional indexes needed for query patterns used in §6.10 (Reports) and §6.5 (asset filtering) – flag before they finalize migrations, since backend read patterns (e.g. filtering assets by holder's department) may need a composite index.
 - The atomic asset-tag-generation mechanism (§6.5) needs either a Postgres sequence or a dedicated counter table – confirm which with DB teammate rather than assuming.
-

13. Build Order (backend track, ~8 hours)

Hours	Focus
0–0.5	Scaffold app.ts/server.ts, env config, prisma client, middleware skeletons, response envelope helpers
0.5– 1.5	Auth module complete (signup/login/refresh/me) + RBAC middleware, testable in Postman
1.5– 2.5	Departments, Categories, Employees (incl. role promotion)
2.5– 4.5	Assets + Allocation workflow (tag generation, allocate/transfer/return, history) – highest priority, allocate most time here
4.5– 5.5	Bookings with conflict detection
5.5– 6.5	Maintenance + Audits
6.5– 7.5	Notifications, Socket.IO wiring, Activity Logs, Dashboard summary, Reports
7.5–8	Postman collection finalize, seed script, deploy to Render, smoke test against frontend

If time runs short, cut in this order: Reports depth → Audit auto-scoping (make it manual asset selection instead) → Socket.IO (fall back to polling, tell frontend teammate immediately) → maintenance-forecast heuristic.